

Observer Pattern

Category: Behavioural Design Pattern

1. The Problem

We start with a `WeatherStation` (data source) that directly holds a reference to a `DisplayDevice` and calls it whenever the temperature changes.

```
class WeatherStation {
    private float temperature;
    private DisplayDevice displayDevice; // hardcoded reference

    public void setTemperature(float temp){
        this.temperature = temp;
        notifyDevice();
    }

    public void notifyDevice(){
        displayDevice.showTemp(temperature);
    }
}
```

Issue: `WeatherStation` is *tightly coupled* to `DisplayDevice`. If we want to notify a mobile device, a smartwatch, etc., we'd have to keep adding new fields/constructor params and edit `WeatherStation` every time a new device type appears — poor scalability and flexibility.

2. The Solution — Observer Pattern

Intent: Define a one-to-many dependency between objects so that when one object (the **Subject**) changes state, all its dependents (the **Observers**) are notified and updated automatically — without the subject knowing the concrete type of its observers.

This is also known as the **Publisher-Subscriber (Pub-Sub) pattern**: - The Subject = **Publisher** (publishes state changes) - The Observers = **Subscribers** (receive updates)

3. Key Idea: Program to an Interface

Instead of the subject holding references to concrete device classes, introduce an **Observer interface** with a common `update()` method. The subject maintains a **list of Observer** (not a list of concrete devices) and calls `update()` on each one when state changes.

- Devices don't get "pushed" as individual typed fields anymore.
- The list can grow/shrink dynamically → add methods to **attach** and **detach** observers at runtime.
- Relationship between Subject and Observers is **composition** (observers are held/contained within the subject's list).

4. Two Ways to Implement It

Variant A — Simple version (interface only)

- `Observer` interface → `update(temp)`
- Subject class directly implements `attach()`, `detach()`, `notifyObservers()` itself.

Variant B — Gang of Four (GoF) version (abstract class)

- `Subject` is pulled out as its own **abstract class / interface** containing `attach()`, `detach()`, `notifyObservers()`.
- The concrete data source (e.g. `WeatherStation`) **extends/implements** `Subject` → becomes the **Concrete Subject**.
- `Observer` remains an interface with `update()`.
- Concrete devices (mobile, display, etc.) implement `Observer` → become **Concrete Observers**.

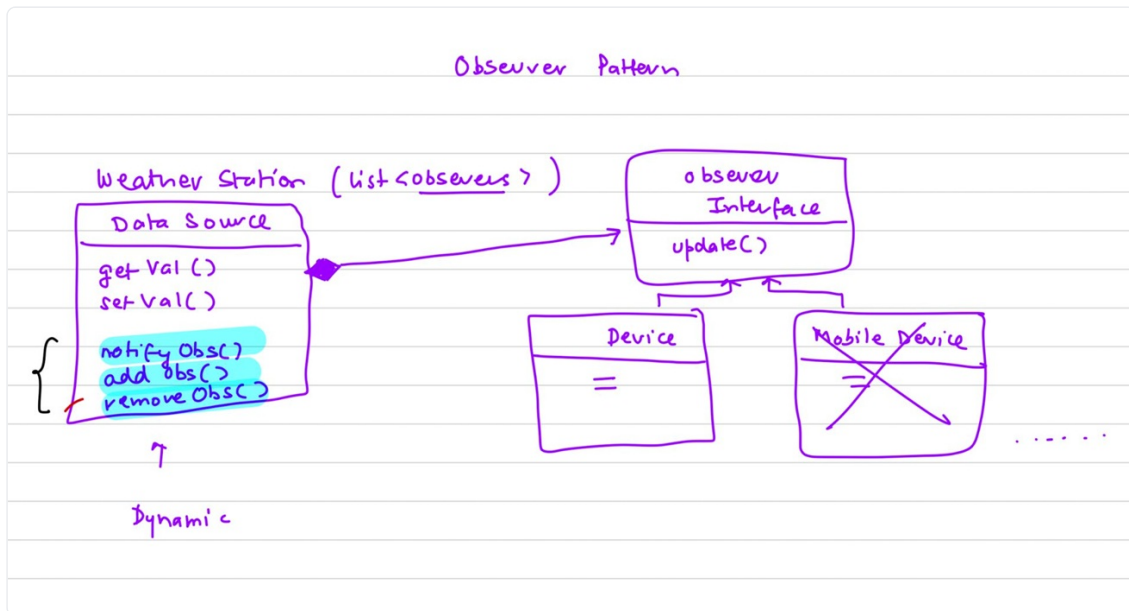
Both variants are functionally the same pattern — GoF's version just abstracts the bookkeeping methods (`attach` / `detach` / `notify`) into a separate `Subject` type so multiple concrete subjects could reuse that contract.

5. Structure / Participants

Role	Responsibility	Example
Subject (interface/abstract class)	Declares <code>attach(Observer)</code> , <code>detach(Observer)</code> , <code>notifyObservers()</code>	<code>Subject</code> interface
Concrete Subject	Holds state, maintains list of observers, calls <code>notifyObservers()</code> when state changes	<code>Weather</code>
Observer (interface)	Declares <code>update(...)</code> method	<code>Observer</code>
Concrete Observer	Implements <code>update()</code> to react to the change	<code>DisplayDevice</code> , <code>MobileDevice</code>

Relationship type: Composition — the Subject *contains* a collection of Observers.

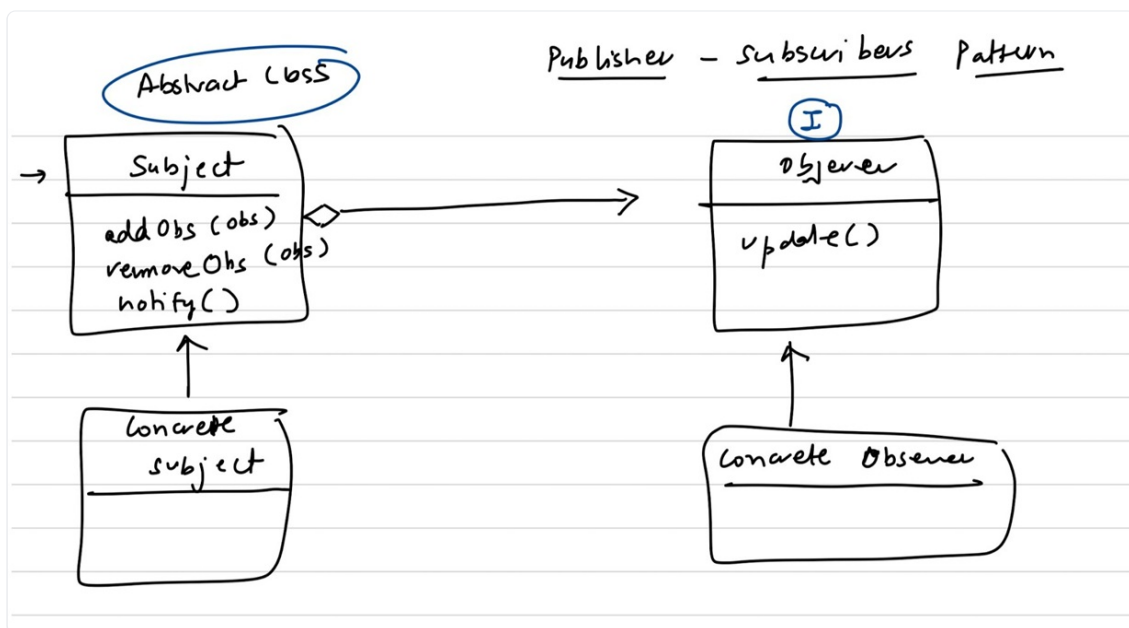
Diagram — Variant A (simple version)



Observer Pattern - simple version diagram

`WeatherStation` (the data source) holds a `List<Observer>` and exposes `notifyObs()`, `addObs()`, `removeObs()` — the diamond denotes composition. The `observer Interface` declares `update()`, which concrete devices (`Device`, `Mobile Device`, ...) implement. Note “Dynamic” — devices can be added/removed from the list at runtime, unlike hardcoded device fields.

Diagram — Variant B (GoF version / Publisher-Subscriber)



Observer Pattern - GoF abstract Subject version diagram

Here the bookkeeping methods (`addObs()`, `removeObs()`, `notify()`) are pulled up into an **abstract subject** class, which `concrete Subject` extends. On the other side, the **observer interface** (marked `I`) declares `update()`, implemented by `concrete observer`. This is the classic GoF structure, also called the **Publisher-Subscriber pattern**.

6. Code — Full Implementation

```

// Observer Interface
interface Observer {
    void update(float temp);
}

// Subject Interface
interface Subject {
    void attach(Observer obs);
    void detach(Observer obs);
    void notifyObservers();
}

// Concrete Subject
class Weather implements Subject {
    private float temperature;
    private List<Observer> observers;

    public Weather() {
        observers = new ArrayList<>();
    }

    public void setTemperature(float temp){
        this.temperature = temp;
        notifyObservers();
    }

    @Override
    public void attach(Observer obs) {
        observers.add(obs);
    }

    @Override
    public void detach(Observer obs) {
        observers.remove(obs);
    }

    @Override
    public void notifyObservers() {
        for (Observer obs : observers) {
            obs.update(temperature);
        }
    }
}

// Concrete Observers
class DisplayDevice implements Observer {
    @Override
    public void update(float temp) {
        System.out.println("Temperature on Display is " + temp);
    }
}

class MobileDevice implements Observer {
    @Override
    public void update(float temp) {
        System.out.println("Temperature on Mobile is " + temp);
    }
}

```

```
// Client
public class ObserverPatternExample {
    public static void main(String[] args) {
        Weather weather = new Weather();
        MobileDevice mobile = new MobileDevice();
        DisplayDevice display = new DisplayDevice();

        weather.attach(mobile);
        weather.attach(display);

        weather.setTemperature(29); // both mobile & display notified
        weather.detach(display);

        weather.setTemperature(2); // only mobile notified now
    }
}
```

What changed vs. the “without pattern” version: - `WeatherStation` (concrete, coupled) → `Weather` implements `Subject` (decoupled) - Fixed `DisplayDevice` field → dynamic `List<Observer>` - No add/remove capability → `attach()` / `detach()` at runtime - Direct method call to one device → loop calling `update()` on all registered observers

7. Benefits

- **Loose Coupling** — the subject (`WeatherStation` / `Weather`) doesn't need to know the concrete type of its observers; it only knows the `observer` interface.
- **Scalability** — new observer types (new display devices, apps, etc.) can be added without touching the subject's code.
- **Flexibility** — observers can be attached/detached dynamically at runtime.

8. Real-World Use Cases

- **Event Listeners** — GUI frameworks notifying listeners on button clicks, input changes, etc.
- **Stock Price Monitoring** — subscribers notified when a stock price changes.
- **News Publishing Systems** — subscribers notified when a new article is published.
- **Social Media Notifications** — followers notified when an account posts.
- **Logging Systems** — multiple handlers (console, file, remote server) observing and logging events.

9. Interview Questions & Answers

Q1. What is the Observer Pattern? A behavioural design pattern that defines a one-to-many dependency between objects, so that when one object (the Subject) changes state, all its registered dependents (the Observers) are notified and updated automatically — without the subject needing to know their concrete types.

Q2. What problem does it solve? It removes tight coupling between a data source and the objects that need to react to changes in it. Without it, the subject would need a hardcoded reference to every dependent type, and adding a new dependent would mean modifying the subject's code.

Q3. What are the main components/participants of the pattern? - `Subject` — declares `attach()`, `detach()`, `notifyObservers()` - `Concrete Subject` — holds state and the list of observers,

triggers notification on state change - `Observer` — declares the `update()` method - `ConcreteObserver` — implements `update()` to react to the change

Q4. What kind of relationship exists between the Subject and its Observers, and why?

Composition — the Subject *contains/maintains* a collection of Observers. The observers' lifecycle (being added/removed) is managed entirely by the subject.

Q5. Why does the Subject depend on an `Observer` interface instead of concrete classes?

This is an application of “program to an interface, not an implementation.” It lets the subject notify any object that implements `Observer`, regardless of its concrete type, keeping the subject decoupled, scalable, and open for extension (Open/Closed Principle).

Q6. How is the Observer Pattern related to the Publisher-Subscriber (Pub-Sub) pattern?

They describe the same core idea: the Subject acts as a **Publisher** that broadcasts state changes, and Observers act as **Subscribers** that receive those updates. “Observer pattern” is the classic GoF name; “pub-sub” is the more general/architectural name for the same concept (and is often used loosely even when there's a message broker involved, which the classic Observer pattern doesn't require).

Q7. How does the GoF (Gang of Four) implementation differ from a simpler custom implementation?

In the GoF version, `attach()`, `detach()`, and `notifyObservers()` are abstracted into a separate `Subject` abstract class/interface, and the concrete data source extends/implements it to become the Concrete Subject. A simpler version just has the concrete subject implement those methods directly. Functionally they achieve the same result — GoF's version just standardizes the contract if multiple concrete subjects need to share it.

Q8. How do you add or remove observers at runtime? Through `attach(Observer obs)` and `detach(Observer obs)` methods on the subject, which add to / remove from its internal list (e.g. `List<Observer>`). This is what makes the observer list dynamic instead of fixed at compile time.

Q9. What are the benefits of using the Observer Pattern? - Loose coupling between subject and observers - Easy scalability — new observer types can be added without changing the subject - Flexibility — observers can be added/removed dynamically at runtime

Q10. What are some drawbacks or things to watch out for? - If not careful, notification chains can create unexpected performance overhead when there are many observers. - Observers are notified in no guaranteed order (unless explicitly managed). - Memory leaks can occur if observers are never detached (the subject keeps holding references to them). - Cascading updates: if an observer's `update()` triggers further state changes, it can lead to complex, hard-to-debug chains of notifications.

Q11. Can you give real-world examples of the Observer Pattern in use? GUI event listeners (button clicks), stock price monitoring, news publishing/subscription systems, social media follower notifications, and logging frameworks with multiple handlers (console, file, remote).

Q12. Is the Observer Pattern push-based or pull-based, and what's the difference? - **Push model:** the subject sends the full changed data directly to observers (e.g. `update(float temp)` — as in this example). - **Pull model:** the subject just notifies observers that *something* changed, and each observer queries the subject for the specific data it needs (e.g. `update(Subject subject)`, then observer calls `subject.getTemperature()`). Push is simpler but can send unnecessary data; pull is more flexible but requires observers to know the subject's interface.